```
import random
import math

# Calculate the number of attacking queen pairs
def calculate_cost(state):
    n = len(state)
    conflicts = 0
    for i in range(n):
        for j in range(i + 1, n):
            if (state[i] == state[j] or
                abs(state[i] - state[j]) == abs(i - j)):
                conflicts += 1
    return conflicts

# Generate a neighboring state
def get_neighbor(state):
    neighbor = state[:]
    col = random.randint(0, 7)
    row = random.randint(1, 8)
    while row == neighbor[col]:
        row = random.randint(1, 8)
    neighbor[col] = row
    return neighbor

# Simulated Annealing algorithm
def simulated_annealing():
    current = [random.randint(1, 8) for _ in range(8)]
    current_cost = calculate_cost(current)
    temperature = 100.0
    cooling_rate = 0.99
    min_temperature = 0.001
    max_iterations = 100000
    for iteration in range(max_iterations):
        if current_cost == 0:
            print("Solution found!")
            print("Final state:", current)
            print("Final cost:", current_cost)
```

```
            print("Iterations:", iteration)
            return current
        neighbor = get_neighbor(current)
        neighbor_cost = calculate_cost(neighbor)
        delta = neighbor_cost - current_cost
        # Accept a better state or sometimes a worse state
        if delta < 0 or random.random() < math.exp(-delta / temperature):
            current = neighbor
            current_cost = neighbor_cost
        # Cool the temperature
        temperature *= cooling_rate
        if temperature < min_temperature:
            temperature = 100.0 # Restart temperature
    print("Solution not found within iteration limit.")
    print("Best final state:", current)
    print("Final cost:", current_cost)
    return current

# Display the chessboard
def display_board(state):
    for row in range(1, 9):
        for col in range(8):
            if state[col] == row:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()

# Execute the algorithm
solution = simulated_annealing()
if calculate_cost(solution) == 0:
    print("\nChessboard:")
    display_board(solution)
```

OUTPUT:

```
Solution found!
Final state: [3, 6, 8, 1, 5, 7, 2, 4]
Final cost: 0
Iterations: 762

Chessboard:
. . . Q . . . .
. . . . . . Q .
Q . . . . . . .
. . . . . . . Q
. . . . Q . . .
. Q . . . . . .
. . . . . Q . .
. . Q . . . . .
```